Aplicación web para la manipulación de paquetes de red aplicado a la unidad de aprendizaje de Redes de Computadoras en ESCOM.

Trabajo Terminal No. 2027-A020

Alumnos: Castillo Bonilla Ángel Eduardo, *García Escamilla Bryan Alexis, Meléndez Macedonio Rodrigo

Directores: Moreno Cervantes Axel Ernesto

*e-mail: bgarciae1900@alumno.ipn.mx

Resumen – Este proyecto propone el desarrollo de una aplicación web para la manipulación de paquetes de red aplicado a la unidad de aprendizaje de Redes de Computadoras en ESCOM. El sistema se basará en la arquitectura TCP/IP para la desencapsulación, análisis y encapsulación de paquetes que usen los protocolos ARP, ICMP, IGMP, DVMRP, PIM, MSDP, TCP y UDP. Su objetivo es sustituir el aprendizaje apoyado únicamente en teoría o herramientas de análisis de tráfico real, que presentan alta complejidad técnica, en respuesta la aplicación ofrecerá un entorno interactivo para la manipulación controlada de paquetes, priorizando la visualización didáctica para facilitar la comprensión del proceso de encapsulación.

Palabras clave – Manipulación de paquetes, aplicación web, análisis de tráfico, encapsulación y desencapsulación de paquetes.

1. Introducción

En el ámbito de la educación en redes de computadoras, uno de los conceptos fundamentales y a la vez más complejos de transmitir es el proceso de encapsulamiento de datos. Este proceso, que permite que la información viaje de manera estructurada y confiable a través de una red, suele explicarse de manera teórica utilizando la arquitectura TCP/IP. Sin embargo, los estudiantes a menudo enfrentan dificultades para visualizar cómo los datos se transforman en cada capa, cómo se construyen las cabeceras y cómo los protocolos de diferentes niveles interactúan para formar una trama lista para ser transmitida.

Tradicionalmente, la enseñanza de estos conceptos se apoya en diagramas estáticos o en herramientas de análisis de tráfico, como Wireshark, que, si bien son poderosas, ofrecen un nivel de detalle que puede resultar abrumador para un principiante. Existe una brecha entre la teoría de los libros y la complejidad de las implementaciones reales, lo que dificulta la comprensión profunda del encapsulamiento y la función específica de cada campo en los protocolos.

Es en este contexto donde surge la necesidad de una herramienta didáctica interactiva que permita a los estudiantes no solo observar, sino también manipular el proceso de encapsulación desde su origen. La comprensión de protocolos como ARP, ICMP, TCP o UDP es crucial, pero su estudio a menudo se limita a su función aislada, sin apreciar cómo estos se integran en las diferentes capas del modelo de referencia.

Para abordar esta problemática, se propone el desarrollo de una aplicación web enfocada en la manipulación de paquetes en la arquitectura TCP/IP. A diferencia de las herramientas convencionales, este sistema permitirá al usuario manipular el contenido de los paquetes capturados que usen los protocolos ARP, ICMP, IGMP, DVMRP, PIM, MSDP, TCP y UDP estudiados en la unidad de aprendizaje de Redes de Computadoras en ESCOM.

La aplicación no solo mostrará el empaquetado final, sino que ofrecerá una vista por capas (arquitectura TCP/IP), permitiendo incluso la manipulación a nivel lógico del contenido del paquete en cualquiera de las capas. Además, contempla la validación de reglas básicas por protocolo, coherencia entre capas, cambios peligrosos o inconsistentes, así como la exportación de paquetes en archivos con el formato estándar PCAP para un estudio posterior más profundo. De esta manera, se busca construir un puente interactivo entre la abstracción teórica y la realidad de los datos en la red.

Incluso más allá de su aplicación en las aulas, esta herramienta posee un potencial significativo en entornos profesionales y de investigación. Para administradores de redes y desarrolladores de sistemas, la capacidad de capturar paquetes y modificar su contenido a nivel lógico ofrece un laboratorio controlado para pruebas de concepto, análisis de vulnerabilidades y validación de comportamientos de red ante condiciones anómalas o malformadas. De esta forma, la aplicación se configura no solo como un recurso pedagógico, sino también como un banco de pruebas versátil para la investigación y el desarrollo en el campo de las redes de computadoras.

Software	Características principales	Precio	Limitaciones	Enfoque
Wireshark	Captura de tráfico en tiempo real con filtros avanzados para análisis. Cuenta con visualización detallada de campos de cabeceras.	Gratuita (Open Source)	No permite generar paquetes desde cero además de requerir tráfico real para funcionar. Presenta una curva de aprendizaje pronunciada. No muestra el proceso de encapsulamiento de forma didáctica.	Análisis forense y depuración de redes para profesionales.
Scapy	Librería de Python para manipulación de paquetes. Puede enviar y capturar tráfico real con soporte de cientos de protocolos.	Gratuita (Open Source)	Requiere programación (no es visual). No muestra el proceso de encapsulamiento gráficamente.	Herramienta de hacking y análisis para expertos que necesitan control total mediante código.
PacketGen	Librería de Ruby similar a Scapy para la generación y manipulación de paquetes.	Gratuita (Open Source)	Requiere conocimientos de Ruby (no es visual) Enfoque en scripting, no en didáctica.	Generación programática de paquetes para pruebas automatizadas.
Spak	Suite de ejecutables independientes. Generación de paquetes Ethernet	Gratuita (Open Source)	Fragmentado (varios programas, sin visualización del proceso).	Herramienta UNIX clásica para construir paquetes de forma secuencial.
BB-Gen	Generador de flujos de paquetes (basado en Scapy) orientado a pruebas de rendimiento.	Gratuita (Open Source)	Solo línea de comandos con un enfoque en rendimiento, no en educación.	Generación masiva de tráfico para validación de redes.
Generador UDP (Yánez Altuna)	Aplicación escritorio (Visual C++) que genera y captura paquetes UDP. Interfaz gráfica básica.	Gratuita (Open Source)	Solo maneja protocolo UDP sin visualización de encapsulamiento. Y tiene enfoque en generación/captura.	Herramienta específica para prácticas con UDP.
Solución propuesta	Aplicación web interactiva con vista por capas de la arquitectura TCP/IP centrado en el manejo de distintos protocolos. Cuenta con manipulación a nivel lógico en cualquier capa con exportación de paquetes con formato PCAP.	Gratuita (Académica)	Alcance limitado a protocolos seleccionados. Entorno simulado.	Combina la potencia de manipulación de Scapy con la claridad pedagógica de un simulador visual, en una plataforma web accesible para educación y pruebas controladas.

Tabla 1. Comparación de soluciones o softwares existentes con la solución propuesta.

2. Objetivo

Objetivo general

Desarrollar una aplicación web para la manipulación de paquetes capturados que usen los protocolos ARP, ICMP, IGMP, DVMRP, PIM, MSDP, TCP y UDP estudiados en la unidad de aprendizaje de Redes de Computadoras en ESCOM.

Objetivos específicos

1. Diseñar e implementar una interfaz web interactiva para la visualización y manipulación lógica de paquetes de red estructurados bajo la arquitectura TCP/IP.
2. Desarrollar una interfaz gráfica que permita visualizar los paquetes de red por capas lógicas de la arquitectura TCP/IP.
3. Implementar una interfaz gráfica para el control y monitoreo del proceso de captura de paquetes de red.
4. Implementar formularios interactivos para la edición lógica de los campos de los paquetes de red.
5. Diseñar una interfaz para la selección y reenvío de paquetes procesados.
6. Implementar una interfaz gráfica para la exportación de paquetes modificados en formato PCAP.
7. Desarrollar un módulo backend para la captura de paquetes de red desde la tarjeta física de red mediante Scapy.
8. Implementar un sistema backend para el almacenamiento y gestión de paquetes de red utilizando formato estándar PCAP.
9. Desarrollar un módulo backend para el procesamiento lógico y desencapsulación de paquetes de red.
10. Implementar reglas backend de validación de coherencia entre capas de la arquitectura TCP/IP.
11. Restringir la modificación de paquetes únicamente a nivel lógico mediante reglas de validación backend.
12. Desarrollar un módulo backend para el reenvío de paquetes procesados hacia la red.
13. Implementar validaciones backend de coherencia mínima del protocolo antes del reenvío de paquetes.
14. Desarrollar un módulo backend para la exportación de paquetes modificados en formato PCAP.
15. Desarrollar una API REST mediante FastAPI para la comunicación entre la interfaz web y el módulo de procesamiento de paquetes.

3. Justificación

El problema por tratar es la desconexión entre la teoría y la práctica en la enseñanza del encapsulamiento de redes, así como la falta de herramientas flexibles para la experimentación controlada con protocolos. El proceso tradicional de aprendizaje se apoya en material teórico estático o en herramientas de análisis de tráfico real como Wireshark, que, aunque poderosas, presentan una curva de aprendizaje pronunciada. Utilizar estas herramientas implica comprender complejos sistemas de filtrado, trabajar con tráfico efímero que no puede ser controlado y revisar visualmente miles de líneas de información lógica, en hexadecimal y ASCII para intentar comprender cómo se construye un paquete desde cero. Este flujo de trabajo hace que la comprensión profunda del encapsulamiento sea lenta, tediosa y poco intuitiva, especialmente para quienes se inician en el campo de las redes.

La propuesta consiste en sustituir el esquema de aprendizaje pasivo y las herramientas de análisis de propósito general por una aplicación web interactiva que permita la captura de paquetes de red desde la tarjeta física de red, su manipulación controlada a nivel lógico, y su posterior reenvío hacía, así como su exportación en formato PCAP. El sistema integrará cinco módulos backend y cinco interfaces web para resolver la brecha educativa y técnica actual.

- **Módulo de captura de paquetes**: Un sistema que capturará paquetes de red que usen los protocolos ARP, ICMP, IGMP, DVMRP, PIM, MSDP, TCP y UDP, usando archivos con el formato estándar PCAP para su almacenamiento.
- **Módulo de almacenamiento de paquetes de red**: Un sistema que guardará mediante índices los paquetes de red capturados usando el gestor de bases de datos no relaciones Mongo DB.
- **Módulo de procesamiento de paquetes**: Un sistema para la desencapsulación/encapsulación de paquetes y para la manipulación de su contenido por capas de la arquitectura TCP/IP a nivel lógico.
- **Módulo de exportación de paquetes:** Un sistema de exportación de paquetes en archivos con el formato estándar PCAP para su análisis independiente.
- **Módulo de reenvío de paquetes:** Un sistema de reenvío de paquetes de forma individual después de su procesamiento.
- **API Rest mediante FastAPI:** Un sistema que valide las peticiones hechas por el usuario desde la interfaz web hacia el modelo de negocio del sistema.
- **Interfaz inicial de la aplicación web:** Una interfaz web para la presentación de la aplicación web con dos opciones: capturar paquetes y manipular paquetes.
- **Interfaz para la captura de paquetes:** Una interfaz web para filtrar el tráfico de red al seleccionar el protocolo, la cantidad máxima de paquetes y el nombre del archivo en formato PCAP para su respectivo almacenamiento, así como para la visualización del número de paquetes capturados y para detener captura.
- **Interfaz para la manipulación de paquetes:** Una interfaz web para la selección del archivo en formato PCAP del cual se tomará la lista de paquetes capturados con una vista por capas TCP/IP, un panel editable a nivel lógico, un botón para guardar cambios y para reenviar paquetes.

- **Interfaz para el reenvío de paquetes:** Interfaz web para selección del paquete a reenviar una vez concluida su manipulación.
- **Interfaz para la exportación de paquetes:** Interfaz web para la exportación de un paquete o múltiples paquetes en archivos en formato PCAP.

La aplicación está dirigida a estudiantes y docentes de la unidad de aprendizaje de Redes de Computadoras en ESCOM, que buscan comprender a fondo el funcionamiento de los protocolos analizados en dicha unidad de aprendizaje. El alcance se limita a la captura y manipulación controlada de paquetes con los protocolos mencionados en un entorno de aplicación web, sin ninguna distinción entre estudiantes y docentes, lo que asegura viabilidad dentro del tiempo y equipo disponibles, sentando las bases para futuras expansiones hacia otros protocolos y funcionalidades.

4. Productos o Resultados esperados

El resultado final de este proyecto será un prototipo de aplicación web funcional para la manipulación de paquetes de red aplicado a la unidad de aprendizaje de Redes de Computadora en ESCOM.

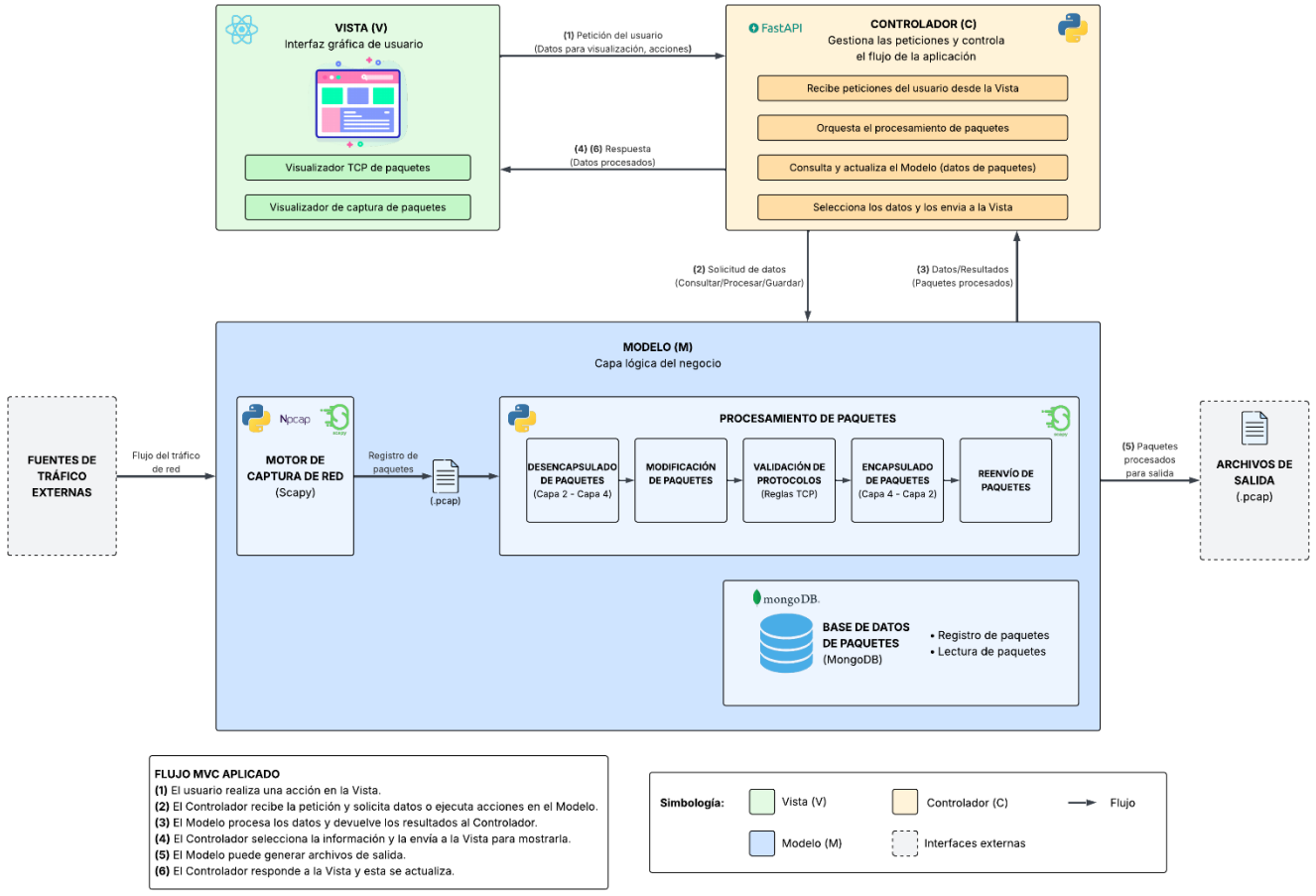


Imagen 1. Arquitectura MVC de la aplicación web para la manipulación de paquetes.

5. Metodología

Para el desarrollo del sistema, se propone la implementación de la metodología en espiral, la cual facilita un enfoque iterativo e incremental. Dada la complejidad técnica del proyecto, este modelo permite abordar cada módulo mediante ciclos de desarrollo compuestos por cuatro fases fundamentales. Al inicio de cada etapa, se planificarán los objetivos específicos y se realizará un análisis de riesgos sobre la implementación de cada componente. Este monitoreo constante es crucial para

identificar desviaciones de manera temprana, permitiendo evaluar alternativas estratégicas que optimicen la calidad del producto final.

Una vez mitigados los riesgos identificados, se procederá a la siguiente fase, que comprende el desarrollo del código fuente y la ejecución de sus respectivas pruebas. En esta etapa se materializan las soluciones evaluadas previamente, para finalmente concluir con una fase de evaluación. Los resultados de dicha evaluación determinarán el rumbo del proyecto y servirán de base para la planificación de la siguiente iteración, repitiendo este ciclo de mejora continua hasta la culminación del sistema. Esta estrategia es fundamental, ya que garantiza un avance controlado, permitiendo gestionar la incertidumbre y ajustar el desarrollo de forma estratégica.

Por otro lado, para asegurar la calidad del prototipo, el sistema se alineará con diversos estándares internacionales. En primer lugar, se adoptará la norma **ISO/IEC 25010** para garantizar que los procesos internos sean precisos, confiables y de alta calidad. Asimismo, la estructuración de la metodología se fundamentará en el estándar **ISO/IEC/IEEE 12207**, asegurando un ciclo de vida del software robusto. Para la validación funcional de los módulos y la verificación de la lógica de red, se empleará la norma **ISO/IEC/IEEE 29119**. Aunado a esto, dado que el sistema manipulará tramas de red a nivel de bits, se integrarán lineamientos de seguridad para el control de datos. Finalmente, se aplicará el estándar **ISO/IEC/IEEE 26511** para el desarrollo de una documentación técnica y de usuario bien estructurada, facilitando la comprensión de los procesos de encapsulamiento.

Iteraciones del desarrollo del sistema

Interfaz inicial de la aplicación web

- **Iteración 1: Panel de bienvenida y enrutamiento del usuario.**

Este ciclo se enfoca en crear las primeras visualizaciones de la página web para iniciar la construcción de todo el sistema de manipulación de datos.

Determinar Objetivos

- Proveer un acceso limpio y directo a la recepción o captura de paquetes red y su posterior procesamiento.
- Establecer la estructura inicial de navegación entre los módulos principales del sistema.

Análisis de Riesgos:

- **Riesgos**: Confusión en la navegación o falta de persistencia en la sesión de trabajo.
- **Mitigación***:* Definir rutas claras y carga inicial del sistema.

Desarrollo y Validación (Subtareas):

- **Backend**:
  - Configurar el enrutador base (FastAPI) para endpoints iniciales.
- **Frontend**:
  - Crear el Dashboard de bienvenida con un diseño moderno (Dark Mode o alguna escala de color que no afecte mucho la vista).
  - Configurar el enrutamiento (ej. React Router) para los flujos: captura y manipulación.

Modelado UML

Se realizará un **Diagrama de Casos de Uso** para representar la interacción inicial del usuario con el sistema y los módulos disponibles.

Justificación: Permite definir de forma clara los puntos de entrada al sistema y la navegación entre módulos principales.

Planificación

- Evaluar la usabilidad antes de construir las pantallas complejas.

Módulo de captura de paquetes

- **Iteración 1: Prototipado Base y Validación de Permisos**

Este ciclo se enfoca en mitigar el riesgo principal: los permisos de la tarjeta de red y la viabilidad de Scapy en segundo plano.

Determinar Objetivos

- Lograr la captura cruda de paquetes desde la interfaz física.

Análisis de Riesgos

- **Riesgo**: Scapy requiere privilegios de root/administrador para interactuar con la tarjeta de red. Si el backend corre sin estos permisos, la app fallará.
- **Mitigación**: Crear un script rápido en consola ejecutado para verificar que Scapy detecta la interfaz física y captura tráfico sin bloquear el flujo principal.

Desarrollo y Validación

- **Backend**:
  - Crear el script base de Scapy para implementar la función de parada/inicio del hilo de captura de forma segura.
- **Frontend**:
  - Crear una vista/dashboard con dos botones: "Iniciar Captura" y "Detener Captura".

Modelo UML

Se elaborará un **Diagrama de Actividad** para representar el flujo general del proceso de captura y control del sistema.

Justificación: Permite visualizar el comportamiento del sistema durante la ejecución del proceso de captura en tiempo real.

Planificación

- Evaluar el rendimiento del script, planificar la estructuración y almacenamiento de los datos capturados para el siguiente ciclo.

- **Iteración 2: Filtrado y Persistencia (Estructuración de Datos)**

Una vez que sabemos que podemos capturar de forma segura, el foco cambia a organizar y guardar esa información.

Determinar Objetivos

- Filtrar los paquetes capturados según los protocolos objetivo (ARP, ICMP, TCP, UDP, etc.).
- Almacenar las capturas tanto en disco (PCAP) como en la base de datos (MongoDB).

Análisis de Riesgos

- **Riesgo**: La inserción en tiempo real de paquetes estructurados en JSON puede saturar la base de datos o consumir demasiada memoria RAM.
- **Mitigación**: Diseñar un mecanismo de almacenamiento temporal para que la escritura en MongoDB no retrase la captura de la tarjeta de red.

Desarrollo y Validación

- **Backend**:
  - Desarrollar el filtro lógico (capas de Scapy) para identificar y separar los protocolos.
  - Implementar la función que vuelca los paquetes crudos a un archivo. pcap.
  - Diseñar el parser que transforma el objeto de Scapy a un documento JSON estructurado y lo inserta en MongoDB.
- **Frontend**:
  - Diseñar una tabla o lista en tiempo real que muestre los paquetes que van llegando (mostrando columnas básicas: IP Origen, IP Destino, Protocolo, Tamaño).
  - Añadir un selector en la interfaz para aplicar los filtros de protocolo desde la vista antes de iniciar la captura.

Modelado UML

Se desarrollará un **Diagrama de Clases** para representar la estructura del sistema de procesamiento de paquetes, desde la captura hasta la persistencia.

Justificación: Permite definir la arquitectura lógica del procesamiento de datos y la separación de responsabilidades.

Planificación

- Diseñar formalmente las interfaces del procesamiento y desencapsulación de los paquetes de red.

Módulo de procesamiento de paquetes de red

➢ Iteración 1: Desencapsulación Base y Visualización Estructurada

Este ciclo se enfoca en mitigar el riesgo de latencia y viabilidad al leer los archivos PCAP y almacenarlos en MongoDB para renderizarlos en la interfaz web por capas.

Determinar Objetivos

- Abrir archivos PCAP y/o consultar los JSON de MongoDB generados en el módulo anterior.

- Desencapsular los paquetes en sus capas lógicas bajo el modelo TCP/IP (Enlace, Red, Transporte, Aplicación).
- Mapear esta estructura de forma interactiva en la web.

Análisis de Riesgos

- **Riesgo:** Cargar un PCAP pesado o un archivo JSON mapeados directamente al frontend puede congelar el navegador.
- **Mitigación:** El servidor solo procesará y enviará un numero paquetes limitado a la vez.

Desarrollo y Validación

- **Backend:**
    - Desarrollar el núcleo del procesador de paquetes utilizando Scapy para abrir los PCAP y separar los headers de cada capa.
    - Validar que el parseo sea 100% fiel al paquete original (hacer un match bit a bit entre el binario crudo y el objeto estructurado).
- **Frontend:**
    - Desarrollar la interfaz base para cargar el PCAP o seleccionar una sesión previa de MongoDB.
    - Diseñar un árbol interactivo que permita al usuario desplegar un paquete y ver sus capas TCP/IP expandidas de forma visual.

Modelado UML

Se realizará un **Diagrama de Componentes**, representando la interacción entre el lector de PCAP, el procesador de paquetes y el servicio de entrega de datos al frontend.

Justificación: Permite visualizar la separación entre la lectura, procesamiento y presentación de los datos del paquete.

Planificación

- Planificar las reglas matemáticas para recalcular los parámetros críticos (como Checksums) cuando el usuario intente editar los campos en el siguiente ciclo.

➢ Iteración 2: Motor de Coherencia y Edición de Parámetros

Una vez que los paquetes se muestran correctamente, el foco es permitir la edición interactiva asegurando que los datos sigan siendo lógicos.

Determinar Objetivos

- Permitir al usuario modificar campos del paquete (IPs, Puertos, Flags, Payload) desde la interfaz.

- Definir la estructura de datos interna que recalcule automáticamente parámetros críticos como longitudes y sumas de verificación (*checksums*).

Análisis de Riesgos

- **Riesgo:** Si el usuario modifica la longitud del payload pero no se actualiza el *Length Header* de IP o el *Checksum* de TCP/UDP, el paquete será corrupto y los routers o sistemas operativos intermedios lo descartarán de inmediato sin procesarlo.
- **Mitigación:** Desarrollar un motor de validación en el backend que verifique la coherencia mínima del protocolo antes de dar por buena la edición.

Desarrollo y Validación

- **Backend:**
    - Desarrollar el motor de coherencia mínima. Implementar las funciones de Scapy que recalculan automáticamente el checksum y las longitudes al alterar los campos.
    - Crear modulos de validación que reciban las modificaciones del frontend, las validen y devuelvan si el paquete es "coherente" o tiene errores de estructura.
- **Frontend:**
    - Transformar la vista interactiva de capas en campos editables.
    - Implementar alertas visuales en tiempo real (ej. si un usuario pone una IP inválida o un puerto fuera de rango, el frontend debe bloquear la confirmación).

Modelado UML

Se desarrollará un **Diagrama de Clases**, representando el motor de edición, validación y reconstrucción del paquete.

Justificación: Permite estructurar la lógica de modificación y recalculo de campos críticos del paquete.

Planificación

- Planificar y configurar el entorno aislado para mitigar los riesgos de red antes de pasar a la fase de reenvío

➢ Iteración 3: Entorno Controlado, Pruebas de Estrés y Cierre

Ciclo final del módulo, enfocado en la seguridad del reenvío y la preparación para la exportación final.

Determinar Objetivos

- Realizar pruebas de estrés enviando errores a propósito para comprobar que las validaciones detienen paquetes peligrosos o inválidos.

Análisis de Riesgos

- **Riesgo:** Un error en el reenvío de paquetes modificados de forma incorrecta puede tirar la red local del lugar donde se prueba.

- **Mitigación:** Restringir las pruebas de reenvío estrictamente a una interfaz virtual o a una topología aislada en GNS3, bloqueando la salida a la tarjeta de red física de producción durante esta fase de pruebas.

Desarrollo y Validación

- **Backend:**
    - Implementar el validador final que actúa como clasificador, si un paquete modificado no pasa la validación de coherencia, el sistema prohíbe su paso hacia la interfaz de retransmisión.
- **Frontend:**
    - Diseñar y habilitar la sección de la interfaz para autorizar el reenvío (un botón de "Preparar para Reenvío" que pase el paquete validado al siguiente módulo).

Modelado UML

Se realizará un **Diagrama de Actividad**, representando el flujo de validación, prueba y autorización de reenvío del paquete.

Justificación: Permite modelar el flujo de control previo a la retransmisión y asegurar integridad del proceso.

Planificación

- Diseñar formalmente las interfaces para los dos módulos finales del Trabajo Terminal: el **Módulo de Exportación** (guardar las modificaciones en nuevos archivos PCAP) y el **Módulo de Reenvío de Paquetes** (inyección final a la red)

Módulo de reenvío de paquetes

- **Iteración 1: Integración Física y Confirmación de Éxito**

En este ciclo nos aseguramos de que el paquete pueda ser enviado de manera correcta para la visualización en el frontend.

Determinar Objetivos

- Retransmitir paquetes modificados de manera individual a través de la tarjeta de red.

Análisis de Riesgos

- **Riesgo**: Analizar el riesgo de seguridad en la red local al reenviar paquetes modificados de manera incorrecta.
- **Mitigación**: Implementar un entorno controlado o sandbox local para pruebas de reenvío de paquetes.

Desarrollo y Validación (Subtareas)

- **Backend:**
    - Desarrollar el Módulo de reenvío de paquetes (Visualización de tráfico de red post-procesamiento).
- **Frontend**:

- Implementar la Interfaz para el reenvío de paquetes (Selección del paquete específico a reenviar).
- Manejar estados de carga (loading) y alertas de confirmación para avisar al usuario si el paquete se envió con éxito o si la interfaz falló.

Modelado UML

Se realizará un **Diagrama de Secuencia**, representando el flujo desde la selección del paquete hasta su reenvío en la red.

Justificación: Permite visualizar la interacción entre frontend, backend y tarjeta de red durante el proceso de envío.

Planificación

- Evaluar si el módulo de reenvío interactúa correctamente con el hardware de red. Diseñar la interfaz de exportación de paquetes red (una unidad por exportación) y pulido estético.

Módulo de exportación de paquetes

- **Iteracion 1: Generación del Binario Estándar y Validación Anticorrupción**

Este ciclo se enfoca críticamente en mitigar el riesgo de corrupción del formato PCAP al empaquetar los datos modificados por el usuario.

Determinar Objetivos

- Tomar los datos estructurados (ya sean los paquetes originales o los modificados en los módulos anteriores) y serializarlos de vuelta al formato binario estructurado. pcap.
- Garantizar la total compatibilidad del archivo generado con herramientas estándar de la industria como Wireshark.

Análisis de Riesgos

- **Riesgo:** Si el usuario editó campos lógicos de forma inconsistente en los ciclos previos, la reconstrucción del binario puede generar un archivo PCAP corrupto que Wireshark rechace.
- **Mitigación:** Implementar una capa de validación estricta en el Backend. Antes de escribir en disco, el sistema pasará los objetos modificados por la función nativa wrpcap de Scapy para verificar que el binario se genera limpiamente y sin excepciones de estructura.

Desarrollo y Validación

- **Backend:**
  - Desarrollar el núcleo del módulo de exportación
  - Implementar el pipeline de empaquetado binario usando wrpcap.
- **Frontend:**
  - Diseñar la lógica y los componentes de la interfaz gráfica dedicados a la exportación.

- Integrar de manera limpia los botones de "Descargar PCAP" en las vistas del historial y del procesador interactivo de paquetes.

Modelado UML

Se realizará un **Diagrama de Componentes**, representando el flujo desde los datos estructurados hasta la generación del archivo PCAP final.

Justificación: Permite visualizar la transformación de datos internos hacia un formato estándar exportable.

Planificación

- Planificar la homogenización visual de toda la plataforma orientada a la experiencia de estudiantes de ingeniería y preparar el entorno para las pruebas de integración final de caja negra.

➢ Iteración 2: Homogenización Estética, Pruebas de Caja Negra y Liberación

El ciclo final del proyecto. Aquí se pule la interfaz con enfoque académico, se valida el sistema de punta a punta y se preparan los entregables del sistema completo.

Determinar Objetivos

- Homogenizar la estética visual de toda la aplicación web para que sea intuitiva y didáctica para estudiantes.
- Completar el flujo completo del sistema (Captura, Procesamiento, Reenvío y Exportación) sin errores.
- Liberar la versión estable de la aplicación web y consolidar la documentación técnica.

Análisis de Riesgos

- **Riesgo:** Que existan fallos de integración ocultos (bugs en el paso de variables, pérdida de persistencia entre módulos o caídas de performance tras un uso prolongado).
- **Mitigación:** Ejecutar pruebas de integración de caja negra exhaustivas con el equipo de desarrollo, simulando el comportamiento de un alumno desde que inicia una captura en la tarjeta de red hasta que descarga su reporte modificado en Wireshark.

Desarrollo y Validación

- **Backend:** Optimizar los endpoints de descarga para soportar transferencias de archivos fluidas.
- **Frontend:**
    - Homogenizar estilos (paleta de colores, tipografías, consistencia en modales y alertas) asegurando una UI limpia y profesional enfocada a laboratorios académicos.
    - Asegurar que los componentes de descarga muestren de forma clara el progreso o el éxito de la exportación.

Modelado UML

Se realizará un **Diagrama de Despliegue**, representando la arquitectura final del sistema y la interacción entre sus módulos principales.

Justificación: Permite consolidar la visión global del sistema en su estado final de producción.

Planificación (Cierre del Proyecto)

- Liberación de la versión final en entorno de producción.
- Recopilación, estructuración y entrega de la documentación técnica final del sistema (arquitectura, manual de usuario y diagramas de clases) requerida para la presentación del sistema.

6. Cronograma

Nombre del alumno(a): **Castillo Bonilla Ángel Eduardo** TT No: 2027-A020

Título del TT: Aplicación web para la manipulación de paquetes de red aplicado a la unidad de aprendizaje de Redes de Computadoras en ESCOM.

Actividad	Fechas		JUL	AGO	SEP	OCT	NOV	DIC	ENE	FEB	MAR	ABR	MAY	JUN	JUL
Interfaz inicial de la aplicación web	20/07/26	20/08/26													
Captura de paquetes (Iteración 1)	22/08/26	31/08/26													
Captura de paquetes (Iteración 2)	02/09/26	17/09/26													
Procesamiento de paquetes de red (Iteración 1)	19/09/26	01/10/26													
Presentación y Evaluación TT1	10/11/26	23/11/26													
Procesamiento de paquetes de red (Iteración 2)	03/11/16	01/01/27													
Procesamiento de paquetes de red (Iteración 3)	05/01/27	28/02/27													
Reenvío de paquetes	03/03/27	30/03/27													
Exportación de paquetes (Iteración 1)	05/04/27	30/04/27													
Exportación de paquetes (Iteración 2)	05/05/27	30/05/27													
Consolidación de la documentación técnica	20/07/26	10/06/27													
Presentación y Evaluación TT2	15/06/27	28/06/27													

Nombre del alumno(a): **García Escamilla Bryan Alexis** TT No: 2027-A020

Título del TT: Aplicación web para la manipulación de paquetes de red aplicado a la unidad de aprendizaje de Redes de Computadoras en ESCOM.

Actividad	Fechas		JUL	AGO	SEP	OCT	NOV	DIC	ENE	FEB	MAR	ABR	MAY	JUN	JUL
Interfaz inicial de la aplicación web	20/07/26	20/08/26													
Captura de paquetes (Iteración 2)	02/09/26	17/09/26													
Procesamiento de paquetes de red (Iteración 1)	19/09/26	01/10/26													
Presentación y Evaluación TT1	10/11/26	23/11/26													
Procesamiento de paquetes de red (Iteración 2)	03/11/16	01/01/27													

Exportación de paquetes (Iteración 1)	05/04/27	30/04/27												
Exportación de paquetes (Iteración 2)	05/05/27	30/05/27												
Consolidación de la documentación técnica	20/07/26	10/06/27												
Presentación y Evaluación TT2	15/06/27	28/06/27												

Nombre del alumno(a): **Melendez Macedonio Rodrigo** TT No: 2027-A020

Título del TT: Aplicación web para la manipulación de paquetes de red aplicado a la unidad de aprendizaje de Redes de Computadoras en ESCOM.

Actividad	Fechas		JUL	AGO	SEP	OCT	NOV	DIC	ENE	FEB	MAR	ABR	MAY	JUN	JUL
Captura de paquetes (Iteración 1)	22/08/26	31/08/26													
Presentación y Evaluación TT1	10/11/26	23/11/26													
Procesamiento de paquetes de red (Iteración 3)	05/01/27	28/02/27													
Reenvío de paquetes	03/03/27	30/03/27													
Exportación de paquetes (Iteración 2)	05/05/27	30/05/27													
Consolidación de la documentación técnica	20/07/26	10/06/27													
Presentación y Evaluación TT2	15/06/27	28/06/27													

7. Referencias

[1] Orebaugh, A., Ramirez, G., & Burke, J. (2007). Wireshark & Ethereal network protocol analyzer toolkit. Syngress.

[2] Sanders, C. (2011). Practical packet analysis: Using Wireshark to solve real-world network problems (2nd ed.). No Starch Press.

[3] Biondi, P. (2024). Scapy: Packet crafting for Python (Versión 2.5.0) [Software]. https://scapy.net/

[4] Cesen, F., Patra, P. G., & Rothenberg, C. (2018). BB-Gen: A packet crafter for data plane evaluation. Anais do I Salão de Ferramentas do Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC 2018). https://doi.org/10.5753/sbrc_estendido.2018.14174

[5] Genesse, S. (2023). PacketGen: Network packet generation and manipulation library for Ruby [Software]. https://github.com/sdaubert/packetgen

[6] ISO – International Organization for Standardization. (s. f.). ISO – International Organization for Standardization. https://www.iso.org/home.html

8. Alumnos y Directores

CARÁCTER: Confidencial
FUNDAMENTO LEGAL: Artículo 11 Fracc. V y Artículos 108, 113 y 117 de la Ley Federal de Transparencia y Acceso a la Información Pública.
PARTES CONFIDENCIALES: Número de boleta y teléfono.

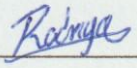
Castillo Bonilla Angel Eduardo – Alumno de la carrera de Ing. en Sistemas Computacionales en ESCOM, Especialidad Sistemas Computacionales, Boleta: 2023630099, Tel. 5514372394, email acastillob1900@alumno.ipn.mx.

Firma 

García Escamilla Bryan Alexis – Alumno de la carrera de Ing. en Sistemas Computacionales en ESCOM, Especialidad Sistemas Computacionales, Boleta: 2023630569, Tel. 5575409913, email bgarciae1900@alumno.ipn.mx.

Firma: 

Meléndez Macedonio Rodrigo – Alumno de la carrera de Ing. en Sistemas Computacionales en ESCOM, Especialidad Sistemas Computacionales, Boleta: 2023630228 Tel. 5547714398, email rmelendezm1900@alumno.ipn.mx.

Firma: 

Moreno Cervantes Axel Ernesto – Dr en Educación (CUGS, 2022). M en C. en Ingeniería Eléctrica (CINVESTAV, 2004). ISC (ESCOM, 2000). Profesor titular Dpto. ISC desde 2004. ext. 52032. Áreas de interés: sistemas distribuidos, redes, email amorenoc@ipn.mx.

Firma: 